

Rapport PPN Semestre 1

Filière : M1 Calcul Haute Performance et Simulation

Années : 2025-2026

Sujet : Path tracing : méthodes de
Monte Carlo pour la synthèse d'images

Rédigé par :
Nolwen DOLLÉANS
Prince Marcel MASSAMBA
Théo BAUER

Encadré par :
Mr. Aurélien DELVAL

Table des matières

1	Path tracing naïf	3
1.1	Rappel de l'estimateur de Monte Carlo et de l'équation de rendu	3
1.2	Implémentation	4
1.3	Premières analyses	4
2	Throughput et roulette russe	5
2.1	Le throughput	5
2.2	Roulette russe	5
2.2.1	État de l'art	5
2.2.2	Implémentation	6
3	Accélération par arbre BVH	6
3.1	Etat de l'art	6
3.2	Implémentation	6
3.2.1	Construction de l'arbre	6
3.2.2	Parcours de l'arbre durant pour l'intersection la plus proche	6
3.3	limites	7
4	Accélération par clustering des K-moyennes	7
4.1	État de l'art	7
4.2	Implémentation de l'arbre BVH par les K-moyennes	7
4.2.1	Détermination des clusters	7
5	SIMD du path tracer naïf	7
5.1	Introduction	7
5.1.1	Vectorisation	8
5.1.2	Vectorisation manuelle	8
5.2	Structure de données	8
5.3	Alignement mémoire	9
5.4	Rayons	9
5.5	Intersections	10
5.6	Échantillonnage du rayon	13
6	Protocole Expérimental	15
6.1	Benchmarks et configuration système	15
6.2	Mesure du runtime	15
6.3	Mesure de la convergence	16
6.3.1	Image de référence	16
6.3.2	Métrique de convergence	16
6.3.3	Déroulement des mesures	16
7	Annexes	16
8	Bibliographie	16

1 Path tracing naïf

1.1 Rappel de l'estimateur de Monte Carlo et de l'équation de rendu

le Path Tracing est une technique de rendu d'image 3D visant à simuler de manière réaliste les rayons lumineux dans une scène. Pour se faire, nous nous sommes basé sur l'équation de rendu, théorisée par James Kajiya en 1986¹ :

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_{\Omega} f(x, \omega_o, \omega_i) L_i(x, \omega_i) \cos(\theta_i) d\omega_i$$

avec :

- $\vec{n}$: La normale de la surface au point x .
- Ω : L'hémisphère centré autour de la normale $\mathbf{n}$.
- x : Le point de $\mathbb{R}^3$ d'intersection entre les rayons incidents et la surface de l'objet considéré.
- $\vec{\omega}_o$: Direction du rayon réfléchi dans Ω sur la surface au point x .
- $\vec{\omega}_i$: Directions des rayons incidents dans Ω sur la surface au point x .
- L_o : La radiance réfléchi sur la surface au point x de direction $\vec{\omega}_o$.
- L_e : La radiance émise par la surface au point x de direction $\vec{\omega}_o$.
- L_i : Les radiances des rayons incidents sur la surface au point x de direction $\vec{\omega}_i$.
- f : Le BRDF qui représente la manière dont le matériaux reflète la lumière.

Cette équation n'étant pas résoluble de manière analytique. Pour se faire, nous utiliserons la méthode Monte Carlo. En effet, cette méthode est la seule méthode qui passe à l'échelle face à la dimension infinie de l'équation de rendu.

Cette méthode se traduit de la manière suivante :

Soit une fonction $g : \begin{cases} \mathbb{R} \rightarrow \mathbb{R} \\ x \rightarrow g(x) \end{cases}$ et I l'intégrale de g dans Ω . I peut s'exprimer de la manière suivante :

$$I = \int_{\Omega} g(x) d_x \approx \frac{1}{N} \sum_k^N \frac{g(x_k)}{\text{pdf}(x_k)}$$

où x_k est la k -ème variable aléatoire, tirée avec la loi de probabilité de densité $\text{pdf}(x)$. La somme converge en $O\left(\frac{1}{\sqrt{N}}\right)$.

Ainsi, l'estimateur Monte Carlo de l'équation de rendu s'écrit :

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \sum_k^N \frac{f(x, \omega_o, \omega_k) L_i(x, \omega_k)}{\text{pdf}(\omega_k)} \cos(\theta_i) d\omega_i$$

où les directions $\vec{\omega}_k$ sont tirées selon $\text{pdf}(\omega)$. Or $L_i(x, \omega_i)$ est elle-même définie par la même équation de rendu au point d'intersection suivant. L'estimateur est donc récursif : évaluer $_o$ en un point nécessite d'évaluer L_i au point suivant, et ainsi de suite.

Un chemin de longueur d est une séquence de points d'intersection $(x_0, \dots, x_d)$, où x_0 est la caméra, x_k le point d'intersection avec la k -ème primitive du chemin et x_d est la source de lumière. La contribution d'un chemin pour un échantillon sera donc pour un échantillonnage pondéré par le cosinus ($\text{pdf}(\vec{\omega}) = \vec{\omega} \bullet \vec{n} / \pi = \cos(\theta) / \pi$, avec $\vec{n}$ la normal sortante de l'objet intersecté au point d'intersection entre le rayon et cette dernière) et avec seulement des matériaux lambertiens ($f(x, \omega_o, \omega) = \text{albedo} / \pi$) :

$$C = L_e(x_d) \cdot \prod_{k=1}^{d-1} \frac{f(x, \omega_o, \omega_k) \cdot \cos(\theta_k)}{\text{pdf}(\omega_k)} = L_e(x_d) \cdot \prod_{k=1}^{d-1} \text{albedo}_k$$

1. Kay T. L., Kajiya J. T., Ray Tracing Complex Scenes, SIGGRAPH 1986.

En pratique, on tire $N = 1$ chemin par pixel à chaque samples, et on moyenne les contributions sur l'ensemble des samples pour faire converger l'estimateur. Cela nous donnera ainsi la couleur d'un pixel.

Complexité de la méthode par rayon : $O(N)$ pour N primitives

Complexité de la méthode : $O(N \cdot P \cdot n)$ pour N primitives, P pixels et n chemins par pixels

Convergence : lente en $\frac{1}{\sqrt{n}}$

1.2 Implémentation

Les primitives sont rangés dans un tableau de la structure suivante :

```
typedef struct Primitive
{
    void *object;
    Vector position;
    Vector color;
    PRIM_TYPE type;
    material_t m_type;
    float albedo;
} Primitive;
```

Ainsi, pour un pixel, nous ferons l'algorithme suivant pour déterminer la contribution C d'un chemin :

Algorithm 1 Algorithme de la contribution d'un chemin

Require: Rayon r , scène S , caméra cam , profondeur max $d_{\max}$

Ensure: Contribution C

```
Initialiser  $T \leftarrow (1, 1, 1)$ 
Initialiser  $r_{actuel} \leftarrow r$ 
for  $d$  do  $(1, \dots, d_{\max})$ 
    if pas d'intersection de  $r_{actuel}$  avec la scène then
        return couleur de fond de la scène
    end if
    if l'objet est émissif then
         $C \leftarrow intensite \cdot couleur\_objet \cdot T$ 
        return  $L$ 
    end if
    Générer un rayon réfléchi  $r_{new}$  dans l'hémisphère
     $T \leftarrow T \cdot albedo \cdot couleur\_objet$ 
     $r_{actuel} \leftarrow r_{new}$ 
end for
return  $C$ 
```

1.3 Premières analyses

Le flame graphe (cf graphe 1) nous permet de remarquer les différents points chauds :

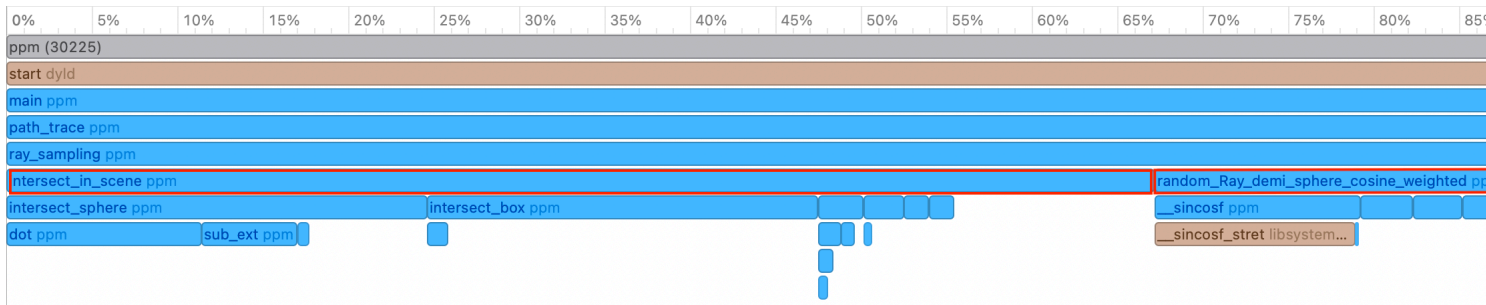


FIGURE 1.1: Flame Graphe du path tracer naïf. On constate 2 gros points chauds (encadrés en rouge)

Ainsi, nous allons focaliser nos améliorations sur la diminution des calculs d'intersections ainsi que du nombre de rayons créés par chemin. Pour ce projet, nous avons sélectionné plusieurs améliorations structurales afin de limiter les calculs d'intersections ainsi que de la création de rayons. Les optimisations que nous avons utilisées sont les suivantes :

- Roulette russe
- Arbre BVH
- Clustering K-Means
- Bidirectional Path Tracing

2 Throughput et roulette russe

2.1 Le throughput

Le throughput se définit comme suit : il représente la contribution cumulée d'un chemin indépendamment de la lumière. Exemple : un rayon qui rebondit sur un mur, puis sur le sol, avant de terminer au niveau de notre œil, va être « modifié » par le mur puis le sol avant d'arriver à l'œil de l'observateur. Le throughput ici va représenter le poids cumulé causé par les interactions avec les objets.

insérer schéma throughput

Ainsi, le throughput T sera calculer de la manière suivante :

Pour un chemin excluant la source de lumière, composé de N primitives :

$$T = \prod_{i=1}^N T_i, \text{ avec } T_i = \text{albedo}_i$$

la contribution d'un chemin s'écrit donc :

$$C = L_e(x_d) \cdot T$$

Comme les matériaux que nous utilisons sont tous lambertiens, le throughput sera toujours le produit de l'albédo de chaque primitive intersectée par le rayon.

2.2 Roulette russe

2.2.1 État de l'art

La roulette russe est une technique issue du transport de particules en physique nucléaire, où elle est utilisée depuis les années 1940 pour simuler le parcours des neutrons dans la matière².

Appliquée à Monte Carlo, cela se traduit comme suit : maintenant que nous avons une donnée quantifiable sur la contribution d'un chemin, on peut décider de stopper les chemins qui contribuent très peu (donc avec un throughput faible). Cependant, simplement supprimer ces chemins introduit un biais en favorisant ceux qui contribuent beaucoup et, de ce fait, provoque une perte d'énergie lumineuse.

2. Arvo J., Kirk D., Particle Transport and Image Synthesis, SIGGRAPH 1990

2.2.2 Implémentation

Pour éviter ce biais, nous choisissons de couper un chemin selon cette condition :

- Nous tirons un nombre aléatoire ϵ
- Si ϵ est supérieur à la composante maximale du throughput (notée p), nous stoppons le chemin
- Si le chemin survit, nous divisons le throughput par p

3 Accélération par arbre BVH

3.1 Etat de l'art

La hiérarchie de volumes englobants (Bounding Volume Hierarchy, BVH) est devenue la structure d'accélération dominante pour le lancer de rayons depuis les années 2000. En effet, elle organise les primitives d'une scène dans un arbre binaire dont chaque nœud stocke une AABB englobant celles de ses descendants^{3 4}.

Ainsi, la construction de l'arbre se fera avec la méthode des centroïdes. Nous allons appeler "centroïdes d'une primitive" le centre de l'AABB la plus petite contenant la primitive. Cette méthode se fait comme suit : à chaque nœud, nous calculons l'AABB des centroïdes des primitives, on choisit comme axe de coupe son axe le plus long, puis on partitionne par médiane. La récursion s'arrête lorsqu'un nœud contient moins de 1 et 4 pour un compromis raisonnable entre profondeur d'arbre et coût des feuilles. La traversée est en $O(\log n)$ en moyenne.

Cette approche a l'avantage de pouvoir sauter des branches entières en fonction de la position de l'AABB englobante du nœud. Exemple : une AABB derrière un rayon ne sera jamais intersectée par ce dernier, il va donc ignorer tous les objets se trouvant à l'intérieur.

3.2 Implémentation

3.2.1 Construction de l'arbre

Jusqu'ici, nous travaillons sur un tableau de structures de primitives. Notre objectif va être de repartir ces primitives dans un arbre BVH décrit dans l'algorithme 1.

insérer algo 1

Cette construction se fait en $O(N \cdot \log N)$, contrairement à celle du tableau en $O(1)$. Cette différence aura un impact important pour un grand nombre de primitives. Cependant, cette perte de performance sera compensée par le parcours de l'arbre en $O(\log N)$ pour les intersection à la place de celle en $O(N)$ pour l'approche naïve. De plus, le temps de l'initialisation de l'arbre est bien négligeable par rapport à celui du rendu de l'image, ce qui compense largement la construction de l'arbre.

3.2.2 Parcours de l'arbre durant pour l'intersection la plus proche

Le rayon est envoyé dans une direction $\vec{d}$ d'une origine O . Pour déterminer l'intersection, nous allons calculer le plus petit réel positif t_{AABB} tel que : $I = O + t_{AABB} \cdot \vec{d}$ avec l'AABB de la racine de l'arbre. S'il n'existe pas, cela veut dire que le rayon n'intersecte pas l'AABB, et donc aucune primitive de l'espace. S'il existe, nous ferons de-même pour les 2 enfants de la racine, et ainsi de suite. Lorsque le rayon intersecte une feuille, nous calculerons le t pour chaque primitive dans la feuille.

L'algorithme s'arrêtera lorsque le rayon aura vérifié l'intersection pour chaque nœud. Le point d'intersection sera donc le t positif le plus petit.

insérer algo 2

3. Rubin S. M., Whitted T., A 3-Dimensional Representation for Fast Rendering of Complex Scenes, SIGGRAPH 1980.

4. Kay T. L., Kajiya J. T., Ray Tracing Complex Scenes, SIGGRAPH 1986.

3.3 limites

Cet algorithme possède des limites. En effet, la partition par médiane sur un seul axe peut produire des AABB qui se chevauchent fortement quand les objets sont distribués irrégulièrement.

Pour palier à ce problème, une solution réside dans le clustering. Pour la suite, nous ferons un clustering par la méthode des K-moyennes.

4 Accélération par clustering des K-moyennes

4.1 État de l'art

Le K-means est l'une des méthodes de partitionnement non supervisé les plus anciennes et les plus utilisées. L'objectif du K-means est de partitionner P en k clusters disjoints $(C_1, \dots, C_k)$, avec k fixé à l'avance ($k \ll N$), tels que chaque point soit affecté à exactement un cluster. Chaque cluster C_j est représenté par son centroïde $\mu_j \in \mathbb{R}^3$.

On cherchera ensuite à résoudre la fonction objective suivante :

$$\min \{J(C_1, \dots, C_k, \mu_1, \dots, \mu_k)\} \text{ tel que : } J = \sum_{j=1}^k \sum_{p \in C_j} \|p - \mu_j\|^2$$

avec p , un centroïde de primitive

L'un des premier algorithme à résoudre cette fonction objective est l'algorithme de Lloyd ⁵, que nous utiliserons dans la suite de ce projet.

4.2 Implémentation de l'arbre BVH par les K-moyennes

4.2.1 Détermination des clusters

Dans un premier temps, nous utiliserons l'algorithme de Lloyd suivant : pour une itération i , on associe chaque centroïde de primitive p au point μ_j le plus proche, déterminée par la norme euclidienne $\|p - \mu_j\|^2$. Ensuite, nous recalculons les centres des clusters C_j , μ_j , comme étant la moyenne de la position de chaque point p associé.

insérer algo 3

Les points μ_j seront initialisés aléatoirement parmi l'ensemble $P = \{p_i\}_{0 \leq i \leq N}$.

Ces k clusters, composé chacun d'un tableau de primitives, seront les points de départ pour former k arbres BVH, décrits précédemment.

Ainsi, nous utiliserons la même méthode pour déterminer les intersections des rayons sur les primitives décrite précédemment, pour chaque BVH construit de cette manière.

Le K-means résout le problème de chevauchement des AABB de l'arbre BVH classique en groupant d'abord les objets spatialement proches avant de construire les BVH locaux.

5 SIMD du path tracer naïf

5.1 Introduction

Dans cette partie, nous allons détailler une implémentation SIMD à la main afin d'identifier d'éventuels gains de performance.

5. Lloyd S. P., Least Squares Quantization in PCM, IEEE Trans. Information Theory, 1982

5.1.1 Vectorisation

Les architectures CPU modernes (x86 ou ARM) permettent deux types d'opérations : les opérations scalaires et les opérations vectorielles.

Code C :

```
float a = 2;
float b = 3;
float c = a + b;
```

Code assembleur :

```
movss xmm0, DWORD PTR .LC0[rip]
movss DWORD PTR [rbp-4], xmm0
```

Code C :

```
for (int i = 0; i < N; ++i)
    c[i] = a[i] + b[i];
```

Code assembleur :

```
.L7:
    movaps xmm0, XMMWORD PTR [rcx+rax]
    addps xmm0, XMMWORD PTR [rdx+rax]
```

5.1.2 Vectorisation manuelle

Il arrive parfois que le compilateur ne puisse pas vectoriser automatiquement certaines boucles. Dans ce cas, on peut utiliser des intrinsics pour forcer une vectorisation.

Code C :

```
# N divisible par 4
for (int i = 0; i < N; i += 4)
{
    __m128 a_vec = _mm_load_ps(a + i);
    __m128 b_vec = _mm_load_ps(b + i);
    __m128 c_vec = _mm_add_ps(a_vec, b_vec);

    _mm_store_ps(c + i, c_vec);
}
```

Code assembleur :

```
.L4:
    vaddps xmm0, xmm0, XMMWORD PTR [rbp-1040+rax]
    vmovaps XMMWORD PTR [rbp-528+rax], xmm0
```

5.2 Structure de données

Notre code de base suit le modèle **Array of Structures** (AoS). Ce modèle peut dégrader la localité des données dans le cache. Pour améliorer la localité mémoire et la vectorisation, on utilise une structure de type **Structure of Arrays** (SoA).

```
typedef struct fSphere
{
    __attribute__((aligned(16))) float *coord;
    __attribute__((aligned(16))) float *x;
    __attribute__((aligned(16))) float *y;
```

```

__attribute__((aligned(16))) float *z;
__attribute__((aligned(16))) float *albedo;
__attribute__((aligned(16))) float *r;
__attribute__((aligned(16))) float *g;
__attribute__((aligned(16))) float *b;
__attribute__((aligned(16))) float *radius;
__attribute__((aligned(16))) float *type;
__attribute__((aligned(16))) float *emission_power;

uint64_t size;
uint64_t capacity;
uint64_t padding;
uint64_t chunked_size;

} fSphere;

```

5.3 Alignement mémoire

`__attribute__((aligned(16)))` indique un alignement mémoire sur 16 octets, nécessaire pour certaines instructions SIMD (SSE).

Cependant, cela n'implique pas que tous les éléments d'un tableau soient alignés individuellement. Pour cela, on utilise une allocation alignée :

```
void *aligned_alloc(size_t alignment, size_t size);
```

On arrondit par un entier multiple de 4 pour éviter des `tail loop`.

[aligned_alloc \(cppreference\)](#)

5.4 Rayons

Le path tracing fonctionne de la manière suivante :

1. Un rayon est lancé depuis la caméra
2. Si le rayon touche une surface, on échantillonne la couleur selon le matériau
3. Le rayon est rebondi jusqu'à une source lumineuse ou un nombre maximal de rebonds

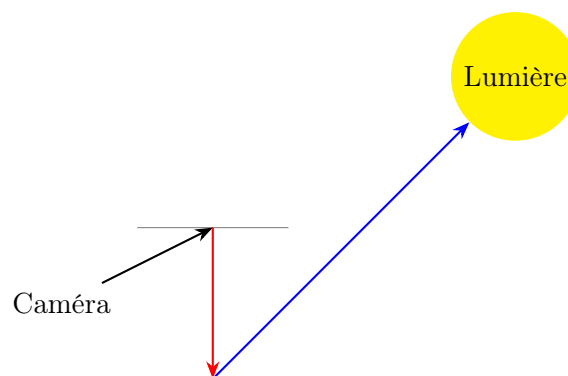


FIGURE 5.1: Path tracing avec rebonds de rayons

Avec SIMD, on traite plusieurs rayons simultanément.

1. Quatre rayons sont lancés depuis la caméra
2. Les intersections sont calculées en parallèle
3. Les rayons sont rebondis simultanément

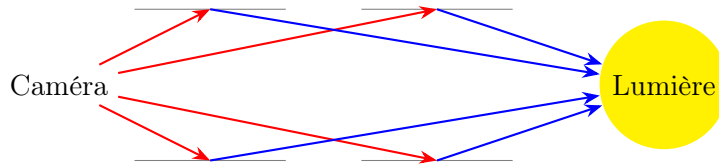


FIGURE 5.2: SIMD path tracing avec 4 rayons traités en parallèle

Ainsi, la structure d'un rayon ressemble à ça :

```
typedef struct vRay
{
    __vec4f ori_x;
    __vec4f ori_y;
    __vec4f ori_z;

    __vec4f dir_x;
    __vec4f dir_y;
    __vec4f dir_z;
} vRay;
```

5.5 Intersections

Le principe du path tracing repose sur la simulation du trajet de la lumière dans une scène 3D. À chaque rebond, il est nécessaire de déterminer si un rayon intersecte une ou plusieurs primitives géométriques, et d'en déduire le point d'impact le plus proche.

Cette étape d'intersection constitue l'un des principaux goulots d'étranglement du rendu, car elle est exécutée un très grand nombre de fois : chaque pixel génère plusieurs rayons, et chaque rayon peut potentiellement être testé contre l'ensemble des objets de la scène.

Dans une implémentation naïve, le coût est donc proportionnel à :

$$O(N_{rays} \times N_{objects})$$

L'objectif de cette section est de présenter une implémentation SIMD de la fonction d'intersection afin de réduire le nombre d'instructions nécessaires par calcul.

L'implémentation SIMD présentée ici repose sur une vectorisation des rayons : quatre rayons sont traités simultanément, tandis que les primitives sont parcourues séquentiellement.

Cette approche permet de conserver une structure simple tout en exploitant efficacement les unités vectorielles du processeur.

```
bool intersect_sphere(Ray* const r, Vector *center, float radius, Vector *hit)
{
    Vector oc;
    sub_ext(&r->position, center, &oc);

    const float A = dot(&r->direction, &r->direction);
    const float B = 2.0f * dot(&r->direction, &oc);
    const float C = dot(&oc, &oc) - radius * radius;

    const float delta = B*B - 4.0f*A*C;

    if (delta < 0.0f)
```

```

        return false;

    float sqrt_delta = sqrtf(delta);
    float t0 = (-B - sqrt_delta) / (2.0f * A);
    float t1 = (-B + sqrt_delta) / (2.0f * A);

    float t = -1.0f;

    if (t0 > EPS)
        t = t0;
    else if (t1 > EPS)
        t = t1;
    else
        return false; # return false if each intersection point is negative

    linear_ext(&r->position, &r->direction, t, hit);
    return true;
}

```

intersect_sphere	ppm	40.8%
intersect_in_scene	ppm	19.7%
intersect_obb	ppm	17.7%
random_Ray_demi_sphere_cosine_weighted	ppm	5.85%
ray_sampling	ppm	5.11%

FIGURE 5.3: Points chauds du code

Principe général :

L'objectif est de vectoriser le calcul d'intersection en traitant quatre rayons simultanément. Contrairement à la version scalaire où un rayon est testé contre toutes les sphères, ici chaque opération SIMD manipule quatre valeurs en parallèle.

Étape 1 : initialisation

```

__vec4f center_x_min = max_limit;
__vec4f center_y_min = max_limit;
__vec4f center_z_min = max_limit;

```

On initialise ici les positions des centres correspondant aux intersections les plus proches. Dans la version scalaire, on cherche la sphère la plus proche pour un rayon. Dans la version SIMD, on effectue cette opération pour quatre rayons simultanément.

Étape 2 : calcul du vecteur origine-centre

```

const __vec4f center_x = set1(sph->x[i]);
const __vec4f center_y = set1(sph->y[i]);
const __vec4f center_z = set1(sph->z[i]);

const __vec4f oc_x = sub_(&packed_ray->ori_x, &center_x);
const __vec4f oc_y = sub_(&packed_ray->ori_y, &center_y);
const __vec4f oc_z = sub_(&packed_ray->ori_z, &center_z);

```

On charge le centre d'une sphère dans les quatre lanes SIMD, puis on calcule le vecteur $\vec{OC}$ entre l'origine des rayons et ce centre.

Étape 3 : produit scalaire

```

const __vec4f h = dot_(&packed_ray->dir_x, &packed_ray->dir_y, &packed_ray->dir_z,
                      &oc_x, &oc_y, &oc_z);

```

On calcule le produit scalaire entre la direction des rayons et le vecteur $\vec{OC}$. Cette valeur est utilisée dans le calcul du discriminant.

Étape 4 : discriminant

```
const __vec4f ac = mul_(&a, &c);
const __vec4f delta = fmsub(&h, &h, &ac);
const __vec4f mask_delta = greater(&delta, &epsilon);
```

Le discriminant $\Delta = h^2 - a \cdot c$ permet de déterminer si une intersection existe. Si $\Delta < 0$, le rayon ne touche pas la sphère.

Étape 5 : test rapide

```
if (fuse(&mask_delta) == 0)
    continue;
```

Si aucun des quatre rayons n'intersecte la sphère, on passe directement à la suivante.

Étape 6 : calcul des solutions

```
const __vec4f fake_delta = max_(&delta, &zero);
const __vec4f sqrt_delta = sqrt_(&fake_delta);

const __vec4f minus_h = sub_(&zero, &h);
const __vec4f t1 = mul_(&sub_(&minus_h, &sqrt_delta), &over_a);
const __vec4f t2 = mul_(&add_(&minus_h, &sqrt_delta), &over_a);
```

On calcule ici les deux solutions de l'équation quadratique correspondant aux points d'intersection.

Étape 7 : sélection des intersections valides

```
const __vec4f test0 = greater(&t1, &epsilon);
const __vec4f test1 = greater(&t2, &epsilon);

__vec4f ts = blendv(&max_limit, &t2, &test1);
ts = blendv(&ts, &t1, &test0);
ts = blendv(&max_limit, &ts, &mask_delta);
```

On filtre les solutions négatives et on sélectionne la plus proche intersection valide.

Étape 8 : mise à jour du hit

```
const __vec4f mask0 = lower(&ts, tmin);

if (fuse(&mask0) == 0)
    continue;
```

```
*tmin = min_(&ts, tmin);
```

On conserve uniquement les intersections les plus proches.

Étape 9 : calcul du point d'impact et des normales

```
const __vec4f hit_point_x = fmadd(&packed_ray->dir_x, tmin, &packed_ray->ori_x);
const __vec4f hit_point_y = fmadd(&packed_ray->dir_y, tmin, &packed_ray->ori_y);
const __vec4f hit_point_z = fmadd(&packed_ray->dir_z, tmin, &packed_ray->ori_z);
```

On calcule la position du point d'impact pour chaque rayon.

Étape 10 : récupération des attributs

Les normales, couleurs et propriétés des matériaux sont ensuite mises à jour à l'aide de masques SIMD afin de ne modifier que les rayons ayant réellement intersecté la sphère.

5.6 Échantillonnage du rayon

Principe général :

L'échantillonnage des rayons permet de simuler les rebonds de la lumière sur les surfaces. Dans le cas d'un matériau diffus (Lambertien), les directions sont tirées aléatoirement dans une demi-sphère orientée selon la normale.

L'implémentation SIMD permet ici de générer quatre directions aléatoires simultanément.

Étape 1 : génération des variables aléatoires

```
const float u01 = (float)rand_r(seed) / (float)RAND_MAX;
...
const float u32 = (float)rand_r(seed) / (float)RAND_MAX;
```

```
__vec4f u_n1 = set(u31, u21, u11, u01);
__vec4f u_n2 = set(u32, u22, u12, u02);
```

On génère des variables aléatoires scalaires, puis on les regroupe dans des vecteurs SIMD afin de traiter quatre échantillons en parallèle.

Étape 2 : conversion en coordonnées sphériques

```
__vec4f atheta = sqrt_(&u_n1);
__vec4f phi = mul_(&two_pi, &u_n2);
```

On transforme les variables aléatoires en coordonnées sphériques adaptées à un échantillonnage cosinus.

Étape 3 : coordonnées locales

```
__vec4f x = cos_(&phi);
__vec4f y = sin_(&phi);
```

```
x = mul_(&x, &atheta);
y = mul_(&y, &atheta);
```

```
__vec4f z = sub_(&one, &u_n1);
z = sqrt_(&z);
```

On obtient une direction dans une base locale où l'axe z correspond à la normale.

Étape 4 : construction d'une base orthonormée

```
__vec4f basic_x = set1(0);
__vec4f basic_y = set1(1);
__vec4f basic_z = set1(0);
```

```
__vec4f right_x = set1(1);
__vec4f right_y = set1(0);
__vec4f right_z = set1(0);
```

On définit des vecteurs permettant de construire une base locale autour de la normale. [aligned_alloc \(opengl-tutorial\)](#)

Étape 5 : calcul des vecteurs tangent et bitangent

```
cross_(normal_x, normal_y, normal_z, &up_x, &up_y, &up_z,
      &tangent_x, &tangent_y, &tangent_z);
```

```
norm_(&tangent_x, &tangent_y, &tangent_z,
     &tangent_x, &tangent_y, &tangent_z);
```

```
cross_(&tangent_x, &tangent_y, &tangent_z,
      normal_x, normal_y, normal_z,
      &bitangent_x, &bitangent_y, &bitangent_z);
```

On construit une base orthonormée (*tangent, bitangent, normal*).

Étape 6 : transformation vers l'espace monde

```
tangent_x = mul_(&tangent_x, &x);
bitangent_x = mul_(&bitangent_x, &y);
__vec4f norm_x = mul_(normal_x, &z);

*dir_x = add_(&tangent_x, &bitangent_x);
*dir_x = add_(dir_x, &norm_x);
```

On combine les trois composantes pour obtenir la direction finale du rayon dans l'espace.

Lancer de rayons (path tracing SIMD)

Principe général :

Cette fonction simule les rebonds successifs d'un paquet de quatre rayons dans la scène. Chaque rayon accumule de la radiance en fonction des matériaux rencontrés (diffus, spéculaire ou émissif).

Étape 1 : initialisation

```
__vec4f throughput_r = one;
__vec4f throughput_g = one;
__vec4f throughput_b = one;
```

Le throughput représente l'énergie transportée par chaque rayon.

Étape 2 : intersection

```
vintersect_in_scene(scene, &current_packed_ray, packed_hit);
__vec4f mask_is_hitting = packed_hit->hit_isHitting;
```

On calcule les intersections pour les quatre rayons.

Étape 3 : gestion des rayons qui ne touchent rien

```
__vec4f mask_is_miss = andnot_(&mask_is_hitting, &mask_is_active);
```

Les rayons qui ne touchent rien récupèrent la couleur de fond.

Étape 4 : mise à jour des rayons actifs

```
mask_is_active = and_(&mask_is_active, &mask_is_hitting);
```

On désactive les rayons qui ne contribuent plus.

Étape 5 : correction de la normale

```
__vec4f dot_normal_ray_dir = dot_(...);
__vec4f mask_is_dot_greater_than_zero = greater(&dot_normal_ray_dir, &zero);
```

On s'assure que la normale est orientée correctement par rapport au rayon incident.

Étape 6 : calcul du point de rebond

```
__vec4f offset_origin_x = add_(&packed_hit->hit_point_x, &hit_surface_nx_eps);
```

On décale légèrement l'origine pour éviter les auto-intersections.

Étape 7 : gestion des matériaux

```
__vec4f mask_is_emissive = equal(...);
__vec4f mask_is_lambertian = equal(...);
__vec4f mask_is_specular = equal(...);
```

Chaque rayon est traité différemment selon le type de matériau rencontré.

Étape 8 : calcul des rebonds

- Diffus : direction aléatoire (échantillonnage cosinus) - Spéculaire : réflexion parfaite - Émissif : arrêt du rayon

Étape 9 : accumulation de la radiance

```
radiance_r = add_(&radiance_r, &emission_throughput_r);
```

On accumule l'énergie transportée par les rayons.

Étape 10 : mise à jour du rayon

```
current_packed_ray.dir_x = blendv(...);
```

On met à jour la direction et la position pour le rebond suivant.

Cette implémentation permet de simuler plusieurs chemins lumineux en parallèle grâce au SIMD.

6 Protocole Expérimental

6.1 Benchmarks et configuration système

Afin de comparer les différentes implémentations, nous avons fait 4 benchmarks différents :

Benchmark **medium** :

- 10 primitives
- 2 sources de lumières
- une bounding box

Benchmark **mickey** :

- 10 primitives
- plusieurs primitives imbriquées
- à "ciel ouvert"

Benchmark **big** :

- 100 primitives
- une bounding box

Benchmark **huge** :

- 1000 primitives
- une bounding box

Les expériences ont été réalisées sur la configuration suivante :

- CPU : Apple Silicon M4, 4 cœurs performance et 6 cœurs efficients
- Compilateur : gcc-15, flags -O3 -march=x86_64v3
- OS : macOS Tahoe 26.4.1

6.2 Mesure du runtime

Pour ce premier type de protocole, nous allons nous concentrer sur le runtime. Il s'agira de mesurer le temps d'exécution de notre programme. Nous avons choisi de mesurer exclusivement la phase de rendu, excluant ainsi la phase d'initialisation, celle de l'écriture de l'image et du calcul du runtime lui-même.

L'objectif sera de comparer le runtime en faisant varier différents paramètres, qui seront explicités pour chaque série de mesures. La scène de référence utilisée est fixée à une résolution de 800×600 pixels, avec [...] primitives, [...] samples et [...] rebonds maximum.

Afin de limiter les incertitudes systématiques, nous avons utilisé la fonction `clock_gettime(CLOCK_MONOTONIC)` de `<time.h>`, qui mesure le temps réel écoulé avec une résolution de 10^{-9} secondes, réduisant ainsi drastiquement l'incertitude de l'outil de mesure. Chaque mesure est répétée 10 fois ; nous retenons la moyenne des répétitions, après suppression des valeurs aberrantes éventuelles.

Critères de comparaison : on considérera qu'une optimisation est bénéfique sur le runtime si le gain mesuré est supérieur à 5%.

6.3 Mesure de la convergence

Pour ce second type de protocole, nous allons mesurer la convergence visuelle du rendu en fonction du nombre de samples. L'objectif est de quantifier l'écart entre une image rendue avec N samples et une image de référence, considérée comme vraie.

6.3.1 Image de référence

L'image de référence est obtenue en rendant la scène avec un nombre de samples N_{ref} suffisamment grand pour que la variance résiduelle soit négligeable devant les écarts que nous souhaitons mesurer. Nous fixons $N_{ref} = 10000$ samples pour chaque benchmarks.

6.3.2 Métrique de convergence

Pour comparer une image rendue I_N à l'image de référence I_{ref} , nous utilisons l'erreur quadratique moyenne (RMSE, *Root Mean Squared Error*) :

$$\text{RMSE}(N) = \sqrt{\frac{1}{3 \cdot W \cdot H} \sum_{i=1}^W \sum_{j=1}^H \sum_{c \in \{R,G,B\}} (I_N(i, j, c) - I_{ref}(i, j, c))^2}$$

où W et H sont respectivement la largeur et la hauteur de l'image en pixels. Le facteur 3 provient de la sommation sur les trois canaux de couleur. Les valeurs de pixels sont normalisées dans $[0, 1]$.

Le RMSE est préféré au MSE car il s'exprime dans la même unité que les valeurs de pixels, facilitant l'interprétation. On s'attend théoriquement à ce que le RMSE décroisse en $O\left(\frac{1}{\sqrt{N}}\right)$, conformément à la convergence de l'estimateur Monte Carlo.

6.3.3 Déroulement des mesures

Pour chaque configuration testée (naïve, BVH, clustering, SIMD), on rend la scène pour $N \in \{10, 50, 100, 200, 500, 1000, 2000\}$ samples. Le script Python calcule le RMSE entre chaque image et I_{ref} , puis trace $\text{RMSE}(N)$ en échelle log-log afin de visualiser la pente de convergence. Une pente de $-\frac{1}{2}$ en log-log confirme la convergence théorique en $O\left(\frac{1}{\sqrt{N}}\right)$.

7 Annexes

8 Bibliographie

Realistic Image Synthesis - Rendering Equation -, par Philipp Slusallek Karol Myszkowski Gurprit Singh : https://graphics.cg.uni-saarland.de/courses/ris-2019/slides/RIS01_RendEq.pdf

Importance Sampling of a Hemisphere, Universität Marburg : <https://www.mathematik.uni-marburg.de/thormae/lectures/graphics1/code/ImportanceSampling/index.html>

Open-GL tutorial - Tutorial 13 - : Normal Mapping: <https://www.opengl-tutorial.org/fr/intermediate-tutorials/tutorial-13-normal-mapping/>

Physically Based Rendering <https://pbr-book.org/4ed/contents>

Generating Camera Rays with Ray-Tracing <https://www.scratchapixel.com/lessons/3d-basic-rendering/ray-tracing-generating-camera-rays/generating-camera-rays.html>

Ray Tracing in One Weekend <https://raytracing.github.io/books/RayTracingInOneWeekend.html>